به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین چهارم

مدرس: مهدی صفرنژاد — انتشار: ۲۷ آبان ۹۸

---

# درهم‌سازی

**زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۸ آذر ۹۸ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد.** به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

سؤال‌های ۱ تا ۳ تئوری و سؤال‌های ۴ تا ۷ عملی هستند. تمامی بخش‌ها را از طریق کوئرا ارسال کنید.

---

**سؤال ۱. [۱۵ نمره]**

فرض کنید یک جدول درهم‌سازی با $m$ خانه داریم که خانه‌های آن را با اندیس‌های $\{۰, ۱, ..., m-۱\}$ شماره گذاری کرده‌ایم. برای یافتن عنصر با کلید $k$ در این جدول مطابق الگوریتم زیر عمل می‌کنیم.

۱. شمارنده‌های $x \leftarrow h(k)$ و $y \leftarrow ۰$ را مقدار دهی کن.

۲. اگر کلید $k$ در خانه $x$اُم جدول بود یا این خانه خالی بود جست‌وجو را متوقف کن.

۳. $x$ و $y$ را به شکل $y \leftarrow (y+۱) \mod m$ و $x \leftarrow (x+y) \mod m$ به‌روز کن و به مرحله‌ی ۲ برگرد.

**آ. [۵ نمره]** ثابت کنید الگوریتم فوق یک نوع وارسی درجه ۲ است.

**ب. [۱۰ نمره]** حال فرض کنید $m = ۲^d$ که $d$ یک عدد صحیح مثبت است. ثابت کنید الگوریتم فوق در بدترین حالت تمام خانه‌های جدول را وارسی می‌کند.

**پاسخ:** آ) در مرحله $i$اُم اجرای الگوریتم فوق خانه‌ی وارسی شده را می‌توان به شکل زیر به‌دست آورد.

$$h(k,i) = x(i) = (h(k) + ۱ + ۲ + ... + i + ۱) \mod m = (h(k) + \frac{i(i+۱)}{۲}) \mod m$$

بنابراین این الگوریتم، یک الگوریتم وارسی درجه دو است.

ب) حالتی که جدول پر است را در نظر بگیرید. کافیست ثابت کنیم تابع $h(k,i)$ با دامنه $۰ \le i \le ۲^d - ۱$ تابعی یک به یک است. (چرا؟) بنابراین فرض کنید $i$ و $j$ به گونه‌ای هستند که $h(k,i) = h(k,j)$. در این صورت

$$h(k,i) = h(k,j) \implies \frac{i(i+۱)}{۲} \equiv \frac{j(j+۱)}{۲} \pmod{۲^d} \implies (i-j)(i+j+۱) \equiv ۰ \pmod{۲^{d+۱}}$$

از طرفی $i \leq ۲^d - ۱$ و $j \geq ۲^d - ۱$ بنابراین $i + j + ۱ \leq ۲^{d+۱} - ۱$. بنابراین $i + j - ۱$ بر $۲^{d+۱}$ بخش‌پذیر نیست. بنابراین

$$i - j \equiv ۰ \pmod{۲^{d+۱}}$$

که با توجه به اینکه $i$ و $j$ هر دو کمتر از $۲^d$ هستند از عبارت فوق می‌توان نتیجه گرفت $i = j$. بنابراین خواسته ما اثبات شد.

**سؤال ۲. [۱۰ نمره]**

آرایه $A$ از اعداد طبیعی داده شده است. می دانیم هر عدد طبیعی حداکثر ۱۰۰۰ بار در آرایه آمده است.

دو نوع کوئری داریم:

۱. کوئری نوع اول این است که برای هر دو عدد طبیعی $n, r$ جایگاه $r$ امین $n$ در دنباله را بگوییم.

۲. کوئری نوع دوم این است که عضوی مانند $i$ را در دنباله عوض کرده و $j$ را قرار دهیم.(تضمین می شود این تغییر شرط اولیه آرایه را نگه می دارد.)

با استفاده از درهم‌سازی داده ساختاری طراحی کنید جواب هر پرسش را در $O(۱)$ بدهد.

**پاسخ:** جواب مسئله یکتا نیست اما یکی از جواب هایی که این پیچیدگی زمانی را به ما می دهد داده ساختار زیر است:

از یک جدول درهم سازی زنجیره ای استفاده می کنیم و روی هر عضو لینک لیست های جدول یک پوینتر به یک $AVL\,TREE$ قرار می دهیم که در هر $AVL\,TREE$ جایگاه های عدد مربوط به آن را قرار می دهیم.

در این صورت برای هر پرسش نوع اول در $O(۱)$ درخت راس پیدا می شود و چون هر درخت حداکثر ۱۰۰۰ راس دارد در $O(۱)$ می توانیم پاسخ پرسش را بدهیم.

به همین صورت چون اضافه کردن و حذف کردن از درخت ها در زمان $O(۱)$ انجام می شود پرسش دوم هم در زمان مطلوب قابل انجام است.

در پاسخ زمان پیدا کردن یک درخت راس را $O(۱)$ در نظر گرفتیم چون نسبت اندازه جدول و اندازه آرایه را میتوان عددی ثابت گرفت و بطور احتمالی طول هر لینک لیست ثابت است.

**سؤال ۳. [۱۵ نمره]**

فرض کنید $U = \{(x_۱, x_۲, \ldots, x_n) \mid x_i \in \{۰, ۱\}\}$ و $V = \{(x_۱, x_۲, \ldots, x_m) \mid x_i \in \{۰, ۱\}\}$ دو فضای برداری روی میدان $\mathbb{Z}_۲$ باشند. جمع برداری و ضرب اسکالر در این میدان همان جمع و ضرب عادی است که به پیمانهٔ ۲ انجام می‌شود.

اگر $m << n$، می‌توانیم تبدیل خطی $T : U \to V$ را یک تابع درهم‌سازی درنظر بگیریم که هر عضو $U$ را به عضوی از $V$ می‌برد. $\mathcal{H}$ را خانوادهٔ تمام تبدیل‌های خطی از $U$ به $V$ بگیرید. ثابت کنید $\mathcal{H}$ سراسری (universal) است.

به عبارت دیگر باید ثابت کنید که اگر $h$ یک عضو $\mathcal{H}$ باشد که به صورت تصادفی با توزیع همگن انتخاب شده است،

$$\Pr(h(x) = h(y)) \leq \frac{۱}{|V|} \qquad \forall x \neq y \in U$$

که $|V| = ۲^m$.

**پاسخ:** هر تابع خطی در $\mathcal{H}$ را می‌توان به صورت یک ماتریس $m \times n$ با درایه‌های صفر و یک نمایش داد. دو بردار $x = (x_۱, x_۲, \ldots, x_n)$ و $y = (y_۱, y_۲, \ldots, y_n)$ در $U$ را در نظر بگیرید که $x \neq y$. می‌خواهیم تعداد ماتریس‌های $T$ با درایه‌های صفر و یک $m \times n$ را بشماریم که $T(x) = T(y)$.

$x \neq y$. در نتیجه $k$ای وجود دارد که $x_k \neq y_k$. ادعا می‌کنیم که در هر حالتی که تمام ستون‌های ماتریس $T$ به غیر از ستون $k$ام، می‌توانند داشته باشند، ستون $k$ام دقیقا یک حالت می‌تواند داشته باشد تا تساوی $T(x) = T(y)$ برقرار شود. فرض کنید $t_{ij}$ درایه سطر $i$ام و ستون $j$ام $T$ باشد. همچنین فرض کنید $T(x) = T(y)$. برای هر $i$، $۱ \leq i \leq n$ داریم:

$$\sum_{j=۱}^{n} t_{ij} x_j = \sum_{j=۱}^{n} t_{ij} y_j$$

بنابراین درایه $i$ام ستون $k$ام برابر است با:

$$t_{ik} = (x_k - y_k)^{-۱} \sum_{j=۱, j \neq k}^{n} t_{ij}(y_j - x_j)$$

ستون $k$ام به طور یکتا مشخص شد. پس از بین تمام $۲^{mn}$ تابع خطی در $\mathcal{H}$، دقیقا $۲^{mn-m} = ۲^{mn}/۲^m$ تابع هستند که $T(x) = T(y)$. اکنون:

$$\Pr(h(x) = h(y)) = \frac{۲^{mn-m}}{۲^{mn}} = \frac{۱}{۲^m} = \frac{۱}{|V|}$$

و حکم مورد نظر ثابت می‌شود.

**سؤال ۴. [۱۵ نمره]**

**در این سؤال استفاده از dictionary مجاز نیست.**

دنیای شنگدباو جای عجیبی است. در این دنیا $n$ نفر زندگی می‌کنند و هر کسی به تعدادی عدد طبیعی حداکثر ۶ رقمی علاقه دارد. اگر به ازای هر عددی که فرد $i$ علاقه دارد فرد $j$نیز به آن علاقه داشته باشد آن وقت فرد $i$ به خود فرد $j$ علاقه خواهد داشت.

شنگدباو از فرد $i$ام در $k_i$ روز یک عدد مورد علاقه او را پرسیده است. او می‌خواهد بداند با اطلاعاتی که الان دارد حداکثر چند نفر وجود دارند که به هم دوطرفه علاقه دارند.

**ورودی:**

در خط اول ورودی عدد $n$ داده می‌شود که نشان دهنده تعداد افراد است. سپس در $n$ خط بعدی در هر خط ابتدا $k_i$ داده می‌شود که تعداد اعدادی است که فرد $i$ام به آن‌ها علاقه دارد و سپس $k_i$ عدد با فاصله داده می‌شود که $j$امین عدد آن عدد مورد علاقه فرد $i$است که در روز $j$ام به شنگدباو گزارش کرده‌است. توجه کنید که ممکن است در این لیست به ازای هر نفر اعداد تکراری هم موجود باشد.

**خروجی:**

در تنها خط خروجی جواب مسئله را چاپ کنید.تنها یک عدد خروجی دهید که برابر تعداد جفت افرادی است که به صورت دوطرفه به هم علاقه دارند.

**محدودیت‌ها:**

$$۱ \leq n \leq ۱۰^۵$$
$$\sum_{i=۱}^{n} k_i \leq ۱۰^۵$$

**ورودی و خروجی نمونه:**

| stdin | stdout |
|---|---|
| 3<br>4 1 10 2 3<br>3 1 2 10<br>5 3 10 1 2 2<br>1 55555 | 1 |

**پاسخ:** برای هر فرد رشته‌ای می‌سازیم از اعداد مورد علاقه‌اش به صورت sort و unique شده که کنار هم چسبیده می‌سازیم. این رشته‌ها را درهم سازی می‌کنیم. سپس به ازای هر مقدار درهم سازی شده می‌توانیم تعداد تکرار هایش را حساب کنیم و به ازای هر رشته، انتخاب ۲ از تعداد تکرار هایش را با هم باید جمع بزنیم.

### سؤال ۵. [۱۵ نمره]

رشته‌ی $s$ متشکل از حروف کوچک انگلیسی به شما داده شده‌است، به چند طریق می‌توان ۲ زیررشته‌ی متفاوت از این رشته انتخاب کرد به نحوی که کاملا برابر باشند.

لازم به ذکر است که زیررشته به معنی چند حرف متوالی از رشته اصلی است.

**ورودی:**
در تنها خط ورودی رشته $s$ به شما داده می‌شود.

**خروجی:**
در تنها خط خروجی جواب مسئله را چاپ کنید.

**محدودیت‌ها:**
$۱ \leq |s| \leq ۱۰۰۰$

**ورودی و خروجی نمونه:**

| stdin | stdout |
|---|---|
| aaa | 4 |

| stdin | stdout |
|---|---|
| aabaab | 10 |

**پاسخ:** برای هر زیر رشته hash آن را بدست می‌آوریم حال کافی است ببینیم چند زیررشته‌ی دیگر همین hash را دارند و این مقدار را به جواب اضافه کنیم.

**سؤال ۶. [۱۵ نمره]**

سید رشته $s$ و $t$ را دارد. او می‌خواهد به ازای هر پسوند رشته $s$ طول بلندترین پیشوند مشترک (LCP) این دو رشته را به دست آورد.

به عبارت دیگر

**پاسخ:**

**سؤال ۷. [۱۵ نمره] بلاک‌چین.** در این سوال شما باید سیستم ساده شده‌ای از بلاک‌چین یا زنجیرهٔ بلوکی را پیاده‌سازی کنید. ابتدا برای آشنایی با بلاک‌چین پیشنهاد می‌شود از این لینک استفاده کنید. سرآیند هر بلوک ۸۰ کاراکتر اول آن را تشکیل می‌دهد و شامل شش فیلد زیر می‌باشد.

| نام فیلد | کاربرد | طول (بایت) |
|---|---|---|
| Version | نسخهٔ بلوک | ۴ |
| hashPrevBlock | درهم‌سازی ۲۵۶ بیتی بلوک قبلی | ۳۲ |
| hashMerkleRoot | درهم‌سازی ۲۵۶ بیتی داده‌های بلوک | ۳۲ |
| Time | زمان حال به ثانیه | ۴ |
| Bits | میزان سختی تولید بلاک | ۴ |
| Nonce | یک عدد دلخواه | ۴ |

بلوک‌ها یک لیست پیوندی را تشکیل می‌دهند که بلاک‌چین یا زنجیرهٔ بلوکی نام دارد. هر بلوک این زنجیره با فیلد hashPrevBlock به بلوک قبلی خود متصل می‌شود. در این سؤال روی یک بلوک از زنجیره تمرکز می‌کنیم. بعد از جمع‌آوری اطلاعات اصلی یک بلوک، سرآیند آن تشکیل می‌شود که ۵ فیلد اول آن مشخص است. برای این که بلاک مجوز اضافه شدن به زنجیره را داشته باشد، باید فیلد ششم یا nonce به صورتی انتخاب شود که اگر تابع SHA-256 را دوبار روی سرآیند بلوک اجرا کنیم، به تعداد خاصی صفر اول آن ظاهر شود. این شرط به این منظور گذاشته شده که اضافه کردن بلوک جدید به زنجیره کار سختی باشد و با وجود تلاش جهانی، حدوداً هر ۱۰ دقیقه یک بلوک به زنجیره اضافه شود. در ورودی به شما ۷۶ کاراکتر اول سرآیند داده می‌شود و وظیفهٔ شما پیدا کردن nonce مناسب و تکمیل بلوک است. به این کار ماینینگ گفته می‌شود. سرآیند در ورودی به صورت هگز (مبنای ۱۶) داده شده است و بنابراین به صورت ۱۵۲ کاراکتر ظاهر می‌شود. در خروجی نیز nonce را باید به صورت هگز بنویسید که ۴ کاراکتر آن به صورت ۸ کاراکتر هگز ظاهر می‌شود. بعد از اضافه کردن nonce، درهم‌سازی سرآیند بلوک باید ۴ تا صفر در نمایش هگز خود داشته باشد (این عدد در واقعیت ۱۸ صفر می‌باشد و به تدریج ممکن است زیاد شود).

توجه کنید ورودی و خروجی به صورت هگز هستند ولی محاسبات باید روی اطلاعات اصلی (باینری) انجام شود. همچنین در تبدیل درهم‌سازی به هگز، ممکن است رشته به صورت برعکس تبدیل شود و صفرها در برخی توابع ابتدا و یا انتهای رشته قرار گیرند. برای همین ابتدا برنامه خود را روی ورودی نمونه تست کنید. شما می‌توانید از کتابخانهٔ hashlib پایتون استفاده کنید. در این لینک بلوک کامل ورودی را می‌توانید پیدا کنید (nonce اصلی این بلوک با خروجی نمونه متفاوت است).

| خروجی | ورودی |
|---|---|
| 7735c84e | 0200000066191da95594aeda1a98a19ff054a88a<br>510754e2a4d93e0a0000000000000008485ae79<br>7312b2cb37dfb1aac11d7c5ad9dd84364bbe26ff<br>a781853996587d9b10a06555f5861618 |

درهم‌سازی ورودی+خروجی نمونه: 0000a22ba24c4bae0cfc072d79b29213364a1b339c1a78fd927d0d3a511231a0

**پاسخ:** در این سؤال کافی است در یک حلقه nonceهای مختلف را بررسی کنید تا جواب پیدا شود. در عین سادگی راه‌حل، این سؤال، الگوریتم استفاده شده در minerهای بلاک‌چین را نشان می‌دهد. در واقع اجرای همین الگوریتم در سال بیش از ۶۶ تراوات‌ساعت برق مصرف می‌کند که بیشتر از مصرف برق بسیاری از کشورها مثل پرتغال است. لازم به ذکر است minerها برای اجرای بهتر و سریع‌تر این الگوریتم که منجر به کشف بیت‌کوین و کسب ثروت می‌شود، سخت‌افزارهای مخصوصی طراحی و تولید کرده‌اند.

فایل `blockchain_sol.py` شامل کد راه‌حل به عنوان نمونه ضمیمه شده است.

موفق باشید!